

# ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

## DEPARTMENT OF INFORMATICS

### ATHANASIOS GOURDOMICHALIS

#### Operating Systems

## 1. Introduction

**i** The **shell** is a program that reads commands from the keyboard and spawns **processes** to execute them. Whenever you type a command into a Unix system terminal (e.g., **ls**, **ps**, **date**, **wc**, etc.), the program responsible for parsing your input and executing the corresponding commands is essentially the shell.

Typically, shells offer various additional functionalities, such as executing scripts (via a built-in interpreter), managing environment variables, and more. In this assignment, the objective is to implement a minimalistic shell whose sole purpose will be to read commands from standard input (keyboard) and execute them.

## 2. Shell Implementation

**i** You are required to implement three shells of increasing complexity and difficulty. These shells must be named **mysh1** through **mysh3**. The three shells should support the following functionalities:

1. **mysh1** reads the name of a program from standard input and then executes it. Note that the program to be executed can reside in any directory listed in the **\$PATH** environment variable.

To execute it, the shell spawns a new process (child process), which runs the specified program. The shell (parent process) waits for the child process to complete execution before accepting the next command. For instance, if you type the command "**ls**" at the **mysh1** prompt, it should execute the familiar **/bin/ls** executable and display the list of files in the current working directory on the screen (more precisely, to **standard output**).

A special case is the “**exit**” command, which does not correspond to an executable program, but simply terminates **mysh1**.

To signal that it is ready to receive the next command, every shell displays a **prompt** on the screen. For this assignment, the prompt for **mysh1** must strictly be “**\$**”, that is, the *dollar sign* followed by *a single space*, with no extra characters before or after.

2. **mysh2** will be an extension of **mysh1**. Specifically, beyond executing programs based solely on their name, it must also support command-line arguments that will be passed to the executed program.

For example, your new shell should support commands such as “**ls -l /tmp**”.

Additionally, **mysh2** must support the built-in **cd** command, which should handle both **absolute paths** (e.g., **cd /home/user**) and **relative paths** (e.g., **cd ../Pictures**).

For implementing **cd**, you should read the manual pages for **chdir**(2) and **getcwd**(3).

3. **mysh3** is a further extension of **mysh2**. It adds support for **pipes**, such as the command “**ls /tmp | wc -l**”. You can assume that the commands will contain at most one pipe-meaning you do not need to support commands with two or more pipes, such as “**sort foo | uniq -c | wc -l**”.

- When implementing **pipes**, you will need to use the **dup**(2) system call, or alternatively **dup2**(2). Refer to the manual pages (**man dup**) for their exact behavior and usage. To **parse** the command entered by the user, you may find **strtok**(3) helpful, though its use is not mandatory.

- Obviously, using the **system()** function is strictly forbidden in this assignment. Equally obvious, invoking another shell (e.g., **/bin/sh**) to do the work on our behalf is also prohibited ;-).

- In the event of invalid input-for instance, if the entered command does not correspond to an existing executable-your shell must not terminate or hang. In such cases, your shell should not produce any output to **stdout**. If you wish to print an error message (e.g., "**Command not found**" or something similar), this is permitted strictly on **stderr**.